

Práctica de laboratorio

507-EGSánchez-PARP601-CLI-IP-Linux-VLAN.docx

Objetivo de la práctica

Primera parte:

Aprender acerca de la configuración de las interfaces de red Ethernet en un sistema de tipo GNU/Linux. Configuramos en él uno o más interfaces de red sobre los interfaces de red su pila de protocolos TCP/IP, asignando direcciones IP, máscaras, puertas por defecto, rutas, etc. Comprobamos su funcionamiento en cada configuración.

Segunda parte

Aprender sobre la configuración de las interfaces de red con IEEE 802.1Q en un sistema de tipo GNU/Linux (Ubuntu Server).

Inventario de material necesario

Primera parte:

- Se necesitan conocimientos básicos del protocolo básico de nivel de red (IP) y de las funciones que realiza esa capa.
- Instalamos el paquete net-tools en el sistema para disponer de ifconfig y netstat.
- Usaremos una máquina con Ubuntu Server instalado (24.04)

Segunda parte

- Además de los elementos anteriores, será necesario otro Ubuntu Server con una interfaz de red Ethernet (en cada uno)

Ejecución

Primera parte

Una vez tenemos nuestra máquina virtual con Ubuntu Server operativo, instalaremos el paquete de Nettools con el comando “*sudo apt install net-tools*”:

```

ergs@ubs:~$ sudo apt install net-tools
[sudo] password for ergs:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes NUEVOS:
  net-tools
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 9 no actualizados.
Se necesita descargar 204 kB de archivos.
Se utilizarán 811 kB de espacio de disco adicional después de esta operación.
Des:1 http://es.archive.ubuntu.com/ubuntu noble-updates/main amd64 net-tools amd64 2.10-0.1ubuntu4.4 [204 kB]
Descargados 204 kB en 0s (5.521 kB/s)
Seleccionando el paquete net-tools previamente no seleccionado.
(Leyendo la base de datos ... 84628 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../net-tools_2.10-0.1ubuntu4.4_amd64.deb ...
Desempaquetando net-tools (2.10-0.1ubuntu4.4) ...
Configurando net-tools (2.10-0.1ubuntu4.4) ...
Procesando disparadores para man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ergs@ubs:~$ _

```

Ahora podremos usar el comando *“ifconfig”* para mostrar información de nuestra interfaz de red y direcciones

```

ergs@ubs:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.65.133 netmask 255.255.255.0 broadcast 192.168.65.255
    inet6 fe80::20c:29ff:feea:9859 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:ea:98:59 txqueuelen 1000 (Ethernet)
    RX packets 62 bytes 25090 (25.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 60 bytes 7051 (7.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ens34: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.65.134 netmask 255.255.255.0 broadcast 192.168.65.255
    inet6 fe80::20c:29ff:feea:9863 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:ea:98:63 txqueuelen 1000 (Ethernet)
    RX packets 173 bytes 216004 (216.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 147 bytes 10107 (10.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 22 bytes 2435 (2.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 22 bytes 2435 (2.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ergs@ubs:~$

```

En nuestra máquina vemos que tenemos 2 tarjetas de red de nombre ens33 y ens34. Centrándonos en la primera, observamos su ip (192.168.65.133), máscara (255.255.255.0), dirección IPv6, MAC, etc.

Comprobamos esta misma información con el comando *“ip address show”* (o *“ip a”*):

```

ergs@ubs:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:ea:98:59 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.65.133/24 metric 100 brd 192.168.65.255 scope global dynamic ens33
        valid_lft 1534sec preferred_lft 1534sec
    inet 10.0.0.50/24 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feea:9859/64 scope link
        valid_lft forever preferred_lft forever
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:ea:98:63 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.65.134/24 metric 100 brd 192.168.65.255 scope global dynamic ens34
        valid_lft 1535sec preferred_lft 1535sec
    inet6 fe80::20c:29ff:feea:9863/64 scope link
        valid_lft forever preferred_lft forever
ergs@ubs:~$

```

Tenemos en cuenta que, como se observa en esta captura, la interfaz ens33 ya tenía una dirección ip asignada por el DHCP para asegurar la conectividad, pero debido a que una misma interfaz física puede tener múltiples direcciones IP lógicas (conocido como *IP Aliasing*) esto no afecta a los requisitos de la práctica a la hora de añadir otra IP. Si quisiéramos quitarla, usaríamos el comando:

"ip address del dev ens33 192.168.65.133/24".

Asignamos una IP secundaria de prueba a la interfaz y la activamos con los siguientes comandos:

"sudo ip address add 10.0.0.50/24 dev ens33"

"sudo ip link set ens33 up"

Ahora, con el comando *"ip route show"* obtendremos la información de la tabla de encaminamiento de nuestra máquina:

```

ergs@ubs:~$ sudo ip address add 10.0.0.50/24 dev ens33
Error: ipv4: Address already assigned.
ergs@ubs:~$ sudo ip link set ens33 up
ergs@ubs:~$ ip route show
default via 192.168.65.2 dev ens33 proto dhcp src 192.168.65.133 metric 100
default via 192.168.65.2 dev ens34 proto dhcp src 192.168.65.134 metric 100
10.0.0.0/24 dev ens33 proto kernel scope link src 10.0.0.50
192.168.65.0/24 dev ens33 proto kernel scope link src 192.168.65.133 metric 100
192.168.65.0/24 dev ens34 proto kernel scope link src 192.168.65.134 metric 100
192.168.65.2 dev ens33 proto dhcp scope link src 192.168.65.133 metric 100
192.168.65.2 dev ens34 proto dhcp scope link src 192.168.65.134 metric 100
ergs@ubs:~$

```

Recordamos que la mayoría de los cambios que hagamos con ip o route en la configuración de red no se mantendrán si se apaga el dispositivo. Esto es debido a que al encender la máquina nuestro Linux lee un fichero que contiene la configuración de red por defecto que se utiliza. Dicha configuración se encuentra en el fichero `/etc/network/interfaces`, pero en versiones más modernas (como la que estamos usando) se encuentra en un archivo. yaml de netcat en `/etc/netplan/50-cloud.init.yaml`.

```
GNU nano 7.2
network:
  version: 2
  ethernet:
    ens33:
      dhcp4: true
    ens34:
      dhcp4: true
```

Lo editamos de la siguiente forma para asignarle la dirección IP (10.0.0.50/24), máscara (255.255.255.0), la puerta de enlace por defecto (10.0.0.1) y los servidores DNS (8.8.8.8 y 1.1.1.1), todo ello solo en la interfaz de nombre ens33. Además, hemos desactivado el DHCP.

```
network:
  version: 2
  renderer: networkd
  ethernet:
    ens33:
      dhcp4: no
      addresses:
        - 10.0.0.50/24
      routes:
        - to: default
          via: 10.0.0.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
    ens34:
      dhcp4: true
```

Guardamos los cambios dándole a “Ctrl + O” y poniendo el comando “sudo netplan try”

```
ergs@ubs:~$ sudo netplan try
Do you want to keep these settings?

Press ENTER before the timeout to accept the new configuration

Changes will revert in 116 seconds
Configuration accepted.
ergs@ubs:~$
```

De esta manera, los cambios que hemos realizado en el archivo yaml se guardarán y aplicarán a nuestro sistema la próxima vez que encendamos la máquina. Para comprobar que los cambios realizados se han hecho correctamente hacemos un autoping a la dirección 10.0.0.50 y esperamos la respuesta:

```

ergs@ub-srv:~$ ping -c 3 10.0.0.50
PING 10.0.0.50 (10.0.0.50) 56(84) bytes of data.
64 bytes from 10.0.0.50: icmp_seq=1 ttl=64 time=0.134 ms
64 bytes from 10.0.0.50: icmp_seq=2 ttl=64 time=0.048 ms
64 bytes from 10.0.0.50: icmp_seq=3 ttl=64 time=0.033 ms

--- 10.0.0.50 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2063ms
rtt min/avg/max/mdev = 0.033/0.071/0.134/0.044 ms
ergs@ub-srv:~$ _

```

```

ergs@ub-srv:~$ ip route show
default via 10.0.0.1 dev ens33 proto static
10.0.0.0/24 dev ens33 proto kernel scope link src 10.0.0.50
192.168.65.0/24 dev ens37 proto kernel scope link src 192.168.65.134
ergs@ub-srv:~$ _

```

Que haya respondido significa que la pila TCP/IP de esa interfaz está activa.

En cuanto a la monitorización de sockets, usaremos el comando “*ss -tulnp*”. El argumento “*tulnp*” Nos permitirá ver las conexiones TCP (t), UDP (u), ver solo los sockets en estado de escucha o LISTEN (l), mostrarlo como números de puertos en lugar de nombres, y revelar qué proceso está usando ese puerto:

```

ergs@ub-srv:~$ sudo ss -tulnp
[sudo] password for ergs:
Netid      State      Recv-Q     Send-Q      Local Address:Port      Peer Address:Port      Process
udp        UNCONN     0           0            127.0.0.54:53           0.0.0.0:*               users:((("systemd-resolve",pid=671,fd=16))
udp        UNCONN     0           0            127.0.0.53%lo:53        0.0.0.0:*               users:((("systemd-resolve",pid=671,fd=14))
tcp        LISTEN     0           4096         127.0.0.54:53           0.0.0.0:*               users:((("systemd-resolve",pid=671,fd=17))
tcp        LISTEN     0           4096         127.0.0.53%lo:53        0.0.0.0:*               users:((("systemd-resolve",pid=671,fd=15))
ergs@ub-srv:~$ _

```

Con esta captura podemos concluir que el servicio de resolución de activos (DNS) está escuchando a través de la IP 127.0.0.53 Y 127.0.0.54 a través del puerto 53, lo que indica que el sistema tiene activo el resolutor local de systemd, que es el encargado de gestionar las peticiones de DS que configuramos antes de Netplan. Además, que las listas de TCP estén en estado de escucha significa que el servidor está abierto y esperando a recibir peticiones en esas interfaces. Sin embargo, la mayoría de los servicios están escuchando en la dirección de loopback (127.0.0.X), por lo que solo están disponibles los servicios internos para el propio Linux, puesto que no están expuestos todavía hacia el exterior (hacia nuestra red 10.0.0.x).

Segunda parte

En ambas máquinas de Ubuntu Server instalamos el paquete “*sudo apt install vlan*” y cargamos el módulo con el comando “*sudo modprobe 8021q*”

Ubuntu server 1

```
ergs@ubs:~$ sudo apt install vlan
[sudo] password for ergs:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes NUEVOS:
  vlan
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 17 no actualizados.
Se necesita descargar 10,4 kB de archivos.
Se utilizarán 51,2 kB de espacio de disco adicional después de esta operación.
Des:1 http://es.archive.ubuntu.com/ubuntu noble/universe amd64 vlan all 2.0.5ubuntu5 [10,4 kB]
Descargados 10,4 kB en 0s (306 kB/s)
Seleccionando el paquete vlan previamente no seleccionado.
(Leyendo la base de datos ... 84625 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../vlan_2.0.5ubuntu5_all.deb ...
Desempaquetando vlan (2.0.5ubuntu5) ...
Configurando vlan (2.0.5ubuntu5) ...
Procesando disparadores para man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ergs@ubs:~$ sudo modprobe 8021q
[sudo] password for ergs:
ergs@ubs:~$
```

Ubuntu server 2

```
ergs@ubs2:~$ sudo apt install vlan
[sudo] password for ergs:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes NUEVOS:
  vlan
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 17 no actualizados.
Se necesita descargar 10,4 kB de archivos.
Se utilizarán 51,2 kB de espacio de disco adicional después de esta operación.
Des:1 http://es.archive.ubuntu.com/ubuntu noble/universe amd64 vlan all 2.0.5ubuntu5 [10,4 kB]
Descargados 10,4 kB en 0s (162 kB/s)
Seleccionando el paquete vlan previamente no seleccionado.
(Leyendo la base de datos ... 84625 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../vlan_2.0.5ubuntu5_all.deb ...
Desempaquetando vlan (2.0.5ubuntu5) ...
Configurando vlan (2.0.5ubuntu5) ...
Procesando disparadores para man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ergs@ubs2:~$ sudo modprobe 8021q
ergs@ubs2:~$
```

Ahora, para configurar el servidor vamos a crear una interfaz virtual sobre la tarjeta física ens33. La llamaremos vlan10 (VLAN ID 10) y editamos ambos archivos YAML:

Ubuntu server 1

```
GNU nano 7.2
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: no
  vlans:
    vlan10:
      id: 10
      link: ens33
      addresses: [10.0.10.1/24]
```

Ubuntu server 2

```
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: no
  vlans:
    vlan10:
      id: 10
      link: ens33
      addresses: [10.0.10.2/24]
```

Para el Ubuntu server 2, hemos hecho exactamente lo mismo, pero con una IP diferente en el mismo rango:

Verificamos que se haya creado correctamente las Vlan en ambas máquinas con “ip a”

Ubuntu server 1

```
configuration accepted.
ergs@ubs:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:cc:ed:7c brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet6 fe80::20c:29ff:fecc:ed7c/64 scope link
        valid_lft forever preferred_lft forever
3: vlan10@ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 00:0c:29:cc:ed:7c brd ff:ff:ff:ff:ff:ff
    inet 10.0.10.1/24 brd 10.0.10.255 scope global vlan10
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fecc:ed7c/64 scope link
        valid_lft forever preferred_lft forever
ergs@ubs:~$
```

Ubuntu server 2

```
ergs@ubs2:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:73:5b:7e brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet6 fe80::20c:29ff:fe73:5b7e/64 scope link
        valid_lft forever preferred_lft forever
3: v1an10@ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 00:0c:29:73:5b:7e brd ff:ff:ff:ff:ff:ff
    inet 10.0.10.2/24 brd 10.0.10.255 scope global v1an10
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe73:5b7e/64 scope link
        valid_lft forever preferred_lft forever
```

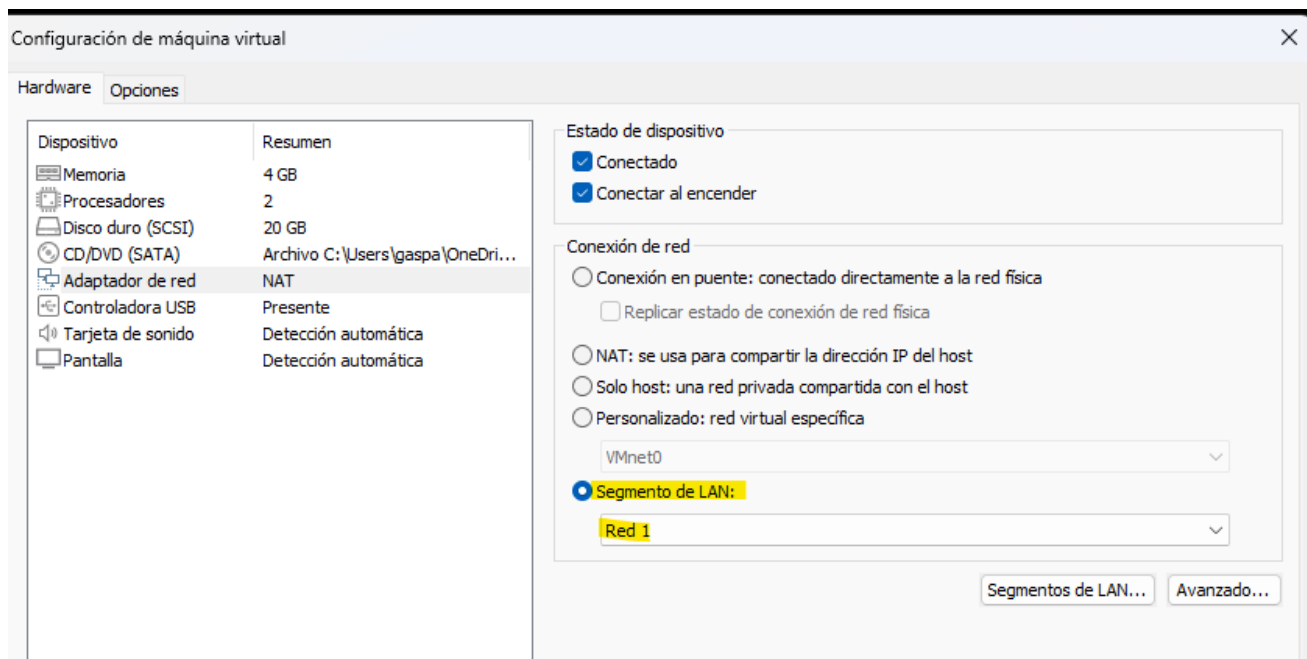
Verificamos el etiquetado con el comando “*ip -d link show v1an10*”, que nos mostrará el Id de la Vlan:

```
ergs@ubs2:~$ ip -d link show v1an10
3: v1an10@ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:cc:ed:7c brd ff:ff:ff:ff:ff:ff promiscuity 0 allmulticast 0 minmtu 0 maxmtu 65535
    vlan protocol 802.1Q id 10 <REORDER_HDR> addrgenmode eui64 numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 tso_max_size 65536 tso_max_segs 65535 gro_max_size 65536
ergs@ubs2:~$
```

Ahora, desde el Ubuntu Server 1, lanzamos el comando “*ping 10.0.10.2*”. Si funciona, significará que el tráfico está viajando encapsulado con el Id 10 por el "cable" virtual de VMware hasta el Ubuntu Server 2.

Para esto nos aseguramos de que ambas máquinas estén en el mismo segmento LAN (Red1)

[Nota: Durante este ping, tendremos el tcpdump a la escucha en Ubuntu Server 2 para confirmar posteriormente que han llegado los paquetes]



```

ergs@ubs:~$ ping 10.0.10.2
PING 10.0.10.2 (10.0.10.2) 56(84) bytes of data.
64 bytes from 10.0.10.2: icmp_seq=1 ttl=64 time=0.921 ms
64 bytes from 10.0.10.2: icmp_seq=2 ttl=64 time=0.443 ms
64 bytes from 10.0.10.2: icmp_seq=3 ttl=64 time=0.389 ms
64 bytes from 10.0.10.2: icmp_seq=4 ttl=64 time=0.399 ms
^C
--- 10.0.10.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3076ms
rtt min/avg/max/mdev = 0.389/0.538/0.921/0.222 ms
ergs@ubs:~$

```

Desde Ubuntu Server 1, el ping ha sido exitoso, y lo podemos confirmar con la siguiente captura de que la dirección su IP se ha comunicado con la de Ubuntu Server 2 mediante el protocolo Vlan activo (802.1Q) de Id 10:

```

ergs@ubs2:~$ sudo tcpdump -i ens33 -e icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
19:40:20.929900 00:0c:29:cc:ed:7c (oui Unknown) > 00:0c:29:73:5b:7e (oui Unknown), ethertype 802.1Q (0x8100), length 102: vlan 10, p 0, ethertype IPv4 (0x0800)
10.0.10.1 > ubs2: ICMP echo request, id 1927, seq 1, length 64
19:40:20.929963 00:0c:29:73:5b:7e (oui Unknown) > 00:0c:29:cc:ed:7c (oui Unknown), ethertype 802.1Q (0x8100), length 102: vlan 10, p 0, ethertype IPv4 (0x0800)
ubs2 > 10.0.10.1: ICMP echo reply, id 1927, seq 1, length 64
19:40:21.931470 00:0c:29:cc:ed:7c (oui Unknown) > 00:0c:29:73:5b:7e (oui Unknown), ethertype 802.1Q (0x8100), length 102: vlan 10, p 0, ethertype IPv4 (0x0800)
10.0.10.1 > ubs2: ICMP echo request, id 1927, seq 2, length 64
19:40:21.931505 00:0c:29:73:5b:7e (oui Unknown) > 00:0c:29:cc:ed:7c (oui Unknown), ethertype 802.1Q (0x8100), length 102: vlan 10, p 0, ethertype IPv4 (0x0800)
ubs2 > 10.0.10.1: ICMP echo reply, id 1927, seq 2, length 64
19:40:22.932254 00:0c:29:cc:ed:7c (oui Unknown) > 00:0c:29:73:5b:7e (oui Unknown), ethertype 802.1Q (0x8100), length 102: vlan 10, p 0, ethertype IPv4 (0x0800)
10.0.10.1 > ubs2: ICMP echo request, id 1927, seq 3, length 64
19:40:22.932286 00:0c:29:73:5b:7e (oui Unknown) > 00:0c:29:cc:ed:7c (oui Unknown), ethertype 802.1Q (0x8100), length 102: vlan 10, p 0, ethertype IPv4 (0x0800)
ubs2 > 10.0.10.1: ICMP echo reply, id 1927, seq 3, length 64
19:40:23.950472 00:0c:29:cc:ed:7c (oui Unknown) > 00:0c:29:73:5b:7e (oui Unknown), ethertype 802.1Q (0x8100), length 102: vlan 10, p 0, ethertype IPv4 (0x0800)
10.0.10.1 > ubs2: ICMP echo request, id 1927, seq 4, length 64
19:40:23.950505 00:0c:29:73:5b:7e (oui Unknown) > 00:0c:29:cc:ed:7c (oui Unknown), ethertype 802.1Q (0x8100), length 102: vlan 10, p 0, ethertype IPv4 (0x0800)
ubs2 > 10.0.10.1: ICMP echo reply, id 1927, seq 4, length 64
19:40:24.974498 00:0c:29:cc:ed:7c (oui Unknown) > 00:0c:29:73:5b:7e (oui Unknown), ethertype 802.1Q (0x8100), length 102: vlan 10, p 0, ethertype IPv4 (0x0800)
10.0.10.1 > ubs2: ICMP echo request, id 1927, seq 5, length 64
19:40:24.974534 00:0c:29:73:5b:7e (oui Unknown) > 00:0c:29:cc:ed:7c (oui Unknown), ethertype 802.1Q (0x8100), length 102: vlan 10, p 0, ethertype IPv4 (0x0800)

```

Consideraciones finales

Esta ha sido una práctica muy realista de la administración de redes en entornos GNU/Linux modernos. He aprendido a gestionar la pila de protocolos TCP/IP mediante 2 maneras; la configuración temporal mediante comandos de consola (como ip address y ss), y la configuración persistente a través de Netplan con la modificación de archivos yaml.

Lo más interesante de la práctica fue la implementación de las VLANs (IEEE 802.1Q), ya que permite experimentar con la segmentación lógica de redes. Comprobar mediante tcpdump que los paquetes viajan con etiquetas específicas y entender que el aislamiento funciona al cambiar los IDs de VLAN fue la parte que más me costó entender.